



PREMIER UNIVERSITY, CHITTAGONG
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Project Report

**Design and Development of a Research Project Management System Using
Laravel, PHP and MySQL**

CSE 4001-INTERNSHIP

Submitted By
MANIK DEY
ID: 0222220005101096
SEMESTER:8th
SECTION:B

Submitted To
IMTIAZ UDDIN RIAD
FOUNDER. TORI IT

Contents

1. Introduction

1.1 Problem Statement

1.2 Objectives

1.3 Project Repository

2. Database Design

2.1 Entity-Relationship (ER) Diagram

2.2 Entity Description

2.3 Database Schema (Migration)

3. Technology Stack

4. Environment Setup

4.1 Installing Laravel and Dependencies

4.2 Database Configuration

4.3 Running Migrations and Seeders

5. Application Architecture (MVC in Laravel)

5.1 Model — ResearchProject

5.2 Controller — ResearchProjectController

5.3 Views (Blade Templates)

5.4 Routes

6. CRUD Functionality

6.1 Create

6.2 Read (List & Details)

6.3 Update

6.4 Delete

7. Additional Features

7.1 Search and Status Filter

7.2 Automatic Slug Generation

7.3 Image Upload

7.4 Pagination

7.5 Form Validation

8. Version Control and Deployment (Git & GitHub)

9. Discussion

10. Conclusion

1. Introduction

1.1 Problem Statement

University research labs and departments typically run several research projects at the same time, each with its own principal investigator, funding source, budget, research area, timeline and current status (upcoming, ongoing or completed). Keeping track of this information in spreadsheets or paper registers becomes difficult as the number of projects grows: records are duplicated, searching for a specific project by title, investigator or research area is slow, and there is no single shareable place from which lab members can view or update project information.

This project addresses that problem by building a small web-based database application, the AG-Lab Research Project Management System, which stores every research project as a structured record in a relational database and exposes full Create, Read, Update and Delete (CRUD) operations through a browser interface. The application was built using the Laravel PHP framework together with a MySQL database, following the Model-View-Controller (MVC) pattern.

1.2 Objectives

- To design a normalized relational database table for storing research project records.
- To build a Laravel-based web application implementing full CRUD (Create, Read, Update, Delete) operations.
- To use Eloquent ORM to map the `research_projects` database table to a PHP model class.
- To provide a searchable, filterable and paginated project listing page.
- To validate all user input on both the create and update forms.
- To support an optional project image upload, stored on the server's public disk.
- To version-control the source code using Git and host the repository on GitHub.
- To document the complete workflow, including practical issues encountered while setting up the environment and Git/GitHub, and how each issue was resolved.

1.3 Project Repository

All source code (models, controllers, views, migrations and seeders) for this project is available at:

<https://github.com/Manikdey96/aqlab-research-projects->

2. Database Design

The application is built around a single core entity, `research_projects`, which stores all the information related to one research project record. Because every field needed by the application (title, description, investigator, funding, dates, status, and an optional image) belongs naturally to one project, a single flat table was sufficient and no additional lookup or join tables were required.

2.1 Entity-Relationship (ER) Diagram

Since every attribute needed by the application belongs to one and the same record (a research project), the system is modelled around a single entity with no foreign-key relationships to other

tables. The Chen-notation diagram below shows the research_projects entity together with all of its attributes; id is the primary key (underlined) and slug carries a unique constraint (dashed outline).



Figure 2.1: Entity-Relationship (ER) diagram of the research_projects entity.

2.2 Entity Description

Field	Type	Description
id	BIGINT (PK, auto-increment)	Unique identifier for each research project record
title	VARCHAR(255)	Title of the research project
slug	VARCHAR(255), UNIQUE	URL-friendly, automatically generated version of the title
description	TEXT	Detailed description of the research project
principal_investigator	VARCHAR(255)	Name of the project's principal investigator (PI)
research_area	VARCHAR(255), NULLABLE	Research area / field of the project (e.g. Plant Abiotic Stress Biology)
funding_source	VARCHAR(255), NULLABLE	Organization funding the project
budget	DECIMAL(12,2), NULLABLE	Approved budget for the project, in BDT
status	ENUM('upcoming','ongoing','completed')	Current status of the project, shown as a colored badge
start_date	DATE	Date on which the project starts / started
end_date	DATE, NULLABLE	Date on which the project ends / ended (blank for ongoing/upcoming projects)

image	VARCHAR(255), NULLABLE	Path to an optional uploaded cover image for the project
created_at / updated_at	TIMESTAMP	Automatically maintained record timestamps

2.3 Database Schema (Migration)

The table structure is defined declaratively using a Laravel migration, which allows the schema to be created, version-controlled and re-created identically on any machine by running a single Artisan command.

```
Schema::create('research_projects', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->string('slug')->unique();
    $table->text('description');
    $table->string('principal_investigator');
    $table->string('research_area')->nullable();
    $table->string('funding_source')->nullable();
    $table->decimal('budget', 12, 2)->nullable();
    $table->enum('status', ['upcoming', 'ongoing', 'completed'])->default('upcoming');
    $table->date('start_date');
    $table->date('end_date')->nullable();
    $table->string('image')->nullable();
    $table->timestamps();
});
```

Using an ENUM column for status keeps the value restricted to exactly the three states the application understands, and a unique index on slug guarantees that no two projects can ever collide on the same URL segment.

3. Technology Stack

Layer	Technology Used
Backend framework	Laravel 13 (PHP 8.3)
Programming language	PHP
Database	MySQL
ORM	Eloquent ORM (Laravel's built-in ORM)
Templating engine	Blade
Frontend styling	Bootstrap 5.3 (via CDN)
Build tooling	Vite, npm
Local development environment	Laragon (Apache/Nginx + MySQL + PHP)
Version control	Git
Remote repository hosting	GitHub
Terminal	Cmder

4. Environment Setup

4.1 Installing Laravel and Dependencies

The project's local development environment was set up using Laragon, which bundles Apache/Nginx, MySQL and PHP and lets an entire local server stack be started or stopped with one

click. Composer, PHP's dependency manager, was used to install Laravel and all of its required packages.

```
composer create-project laravel/laravel aglab-research
cd aglab-research
composer install
npm install
```

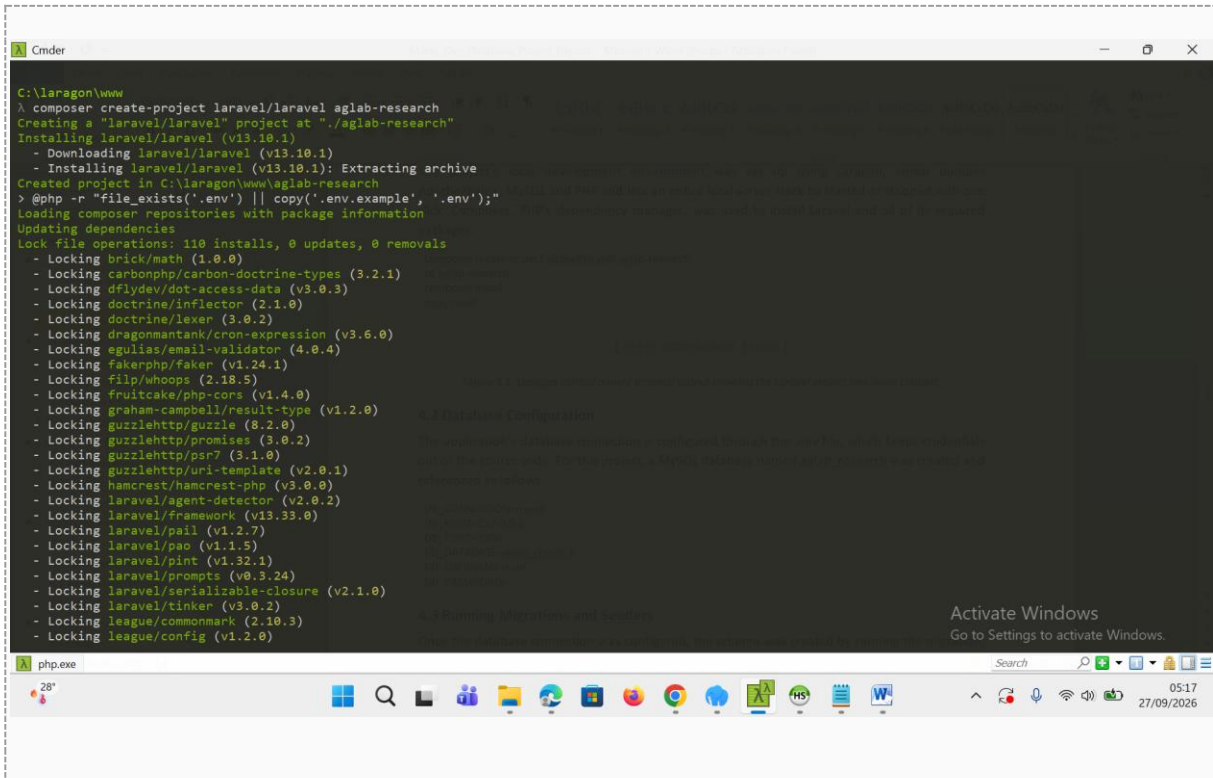


Figure 4.1: Laragon control panel / terminal output showing the Laravel project files being created.

4.2 Database Configuration

The application's database connection is configured through the .env file, which keeps credentials out of the source code. For this project, a MySQL database named `aglab_research` was created and referenced as follows:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=aglab_research
DB_USERNAME=root
DB_PASSWORD=
```

4.3 Running Migrations and Seeders

Once the database connection was configured, the schema was created by running the migration, and the table was populated with a handful of realistic sample records (covering all three status values) using a database seeder, so the CRUD interface could be tested immediately.

```
php artisan migrate
php artisan db:seed --class=ResearchProjectSeeder
php artisan serve
```

```
C:\laragon\www\aglab-research
1 vendor\laravel\framework\src\Illuminate\Database\Connection.php:435
PDOException: ("SQLSTATE[42S02]: Base table or view not found: 1146 Table 'aglab_research.research_projects' doesn't exist")

2 vendor\laravel\framework\src\Illuminate\Database\Connection.php:435
PDO::prepare("select exists(select * from `research_projects` where `slug` = ?) as `exists`")

C:\laragon\www\aglab-research
λ php artisan migrate
INFO Running migrations.
2026_09_27_000001_create_research_projects_table ..... 106.08ms DONE

C:\laragon\www\aglab-research
λ php artisan db:seed --class=ResearchProjectSeeder
INFO Seeding database.
Database\Seeders\ResearchProjectSeeder ..... RUNNING
Database\Seeders\ResearchProjectSeeder ..... 205 ms DONE

C:\laragon\www\aglab-research
λ php artisan serve
INFO Server running on [http://127.0.0.1:8000].
Press Ctrl+C to stop the server

2026-09-27 03:28:49 / ..... ~ 2s
2026-09-27 03:28:52 /favicon.ico ..... ~ 3.35ms
2026-09-27 03:30:16 /research-projects ..... ~ 3s
2026-09-27 03:30:19 /favicon.ico ..... ~ 4.05ms
```

Figure 4.3: Terminal output showing the migration and seeder running successfully, followed by 'php artisan serve' starting the local server.

5. Application Architecture (MVC in Laravel)

The application follows Laravel's Model-View-Controller (MVC) architecture, which keeps database logic, request-handling logic and presentation logic in three separate, clearly defined layers.

5.1 Model — ResearchProject

The ResearchProject Eloquent model represents one row of the research_projects table as a PHP object. Besides declaring which fields are mass-assignable, it also automatically generates a unique URL slug from the project title whenever a new record is created, and exposes a small helper used by the views to color-code the status badge.

```
protected static function booted(): void
{
    static::creating(function (ResearchProject $project) {
        if (empty($project->slug)) {
            $project->slug = static::generateUniqueSlug($project->title);
        }
    });
}

public function statusColor(): string
{
    return match ($this->status) {
        'ongoing' => 'primary',
        'completed' => 'success',
        'upcoming' => 'warning',
        default => 'secondary',
    };
}
```

```
};
}
```

5.2 Controller — ResearchProjectController

A single resource controller, `ResearchProjectController`, implements all seven standard CRUD actions (index, create, store, show, edit, update, destroy). The index method additionally applies an optional keyword search and status filter before paginating the results, and store/update share one private `validateData()` method so the validation rules never drift out of sync between creating and editing a project.

```
private function validateData(Request $request, ?int $ignoreId = null): array
{
    return $request->validate([
        'title' => 'required|string|max:255',
        'description' => 'required|string',
        'principal_investigator' => 'required|string|max:255',
        'research_area' => 'nullable|string|max:255',
        'funding_source' => 'nullable|string|max:255',
        'budget' => 'nullable|numeric|min:0',
        'status' => ['required', Rule::in(['upcoming', 'ongoing', 'completed'])],
        'start_date' => 'required|date',
        'end_date' => 'nullable|date|after_or_equal:start_date',
        'image' => 'nullable|image|mimes:jpg,jpeg,png,webp|max:2048',
    ]);
}
```

5.3 Views (Blade Templates)

The presentation layer is built with five Blade templates, all extending one shared layout (`layouts/app.blade.php`) that provides the green navigation bar and the flash-message banner used across the whole application:

View file	Purpose
<code>index.blade.php</code>	Lists all projects as cards, with a search box, status filter and pagination links
<code>show.blade.php</code>	Displays the full details of one selected project
<code>create.blade.php</code>	Form for adding a new project
<code>edit.blade.php</code>	Form (pre-filled) for updating an existing project
<code>_form.blade.php</code>	Shared partial containing the input fields, included by both create and edit

5.4 Routes

A single `Route::resource()` call registers all seven CRUD routes (index, create, store, show, edit, update, destroy) under the `research-projects` URL prefix, following Laravel's RESTful naming convention:

```
Route::resource('research-projects', ResearchProjectController::class);
```

HTTP Verb	URI	Action	Purpose
GET	<code>/research-projects</code>	index	List / search / filter all projects
GET	<code>/research-projects/create</code>	create	Show the 'add new project' form
POST	<code>/research-projects</code>	store	Save a new project
GET	<code>/research-projects/{id}</code>	show	View one project's details

GET	/research-projects/{id}/edit	edit	Show the 'edit project' form
PUT/PATCH	/research-projects/{id}	update	Save changes to a project
DELETE	/research-projects/{id}	destroy	Delete a project

6. CRUD Functionality

The general flow followed for every write operation is the same: Browser Request → Route → Controller → Validation → Eloquent Model → MySQL Database → Redirect back with a flash message. The four operations are described individually below, each with the corresponding screen from the running application.

6.1 Create

Objective: to add a new research project to the database through a web form.

Clicking '+ New Project' opens the create form, which posts to the store() controller action. After validation succeeds, a new row is inserted into research_projects (with a slug generated automatically from the title), and the user is redirected back to the project list with a success message.

Figure 6.1: The 'Add New Research Project' create form

6.2 Read (List & Details)

Objective: to browse all projects and view the full details of a single project.

The index page (research-projects) lists every project as a card showing its title, status badge, principal investigator and a short excerpt of the description. Clicking 'View' opens the show page for that project, which displays every stored field — including the funding source, budget, start/end dates and full description.

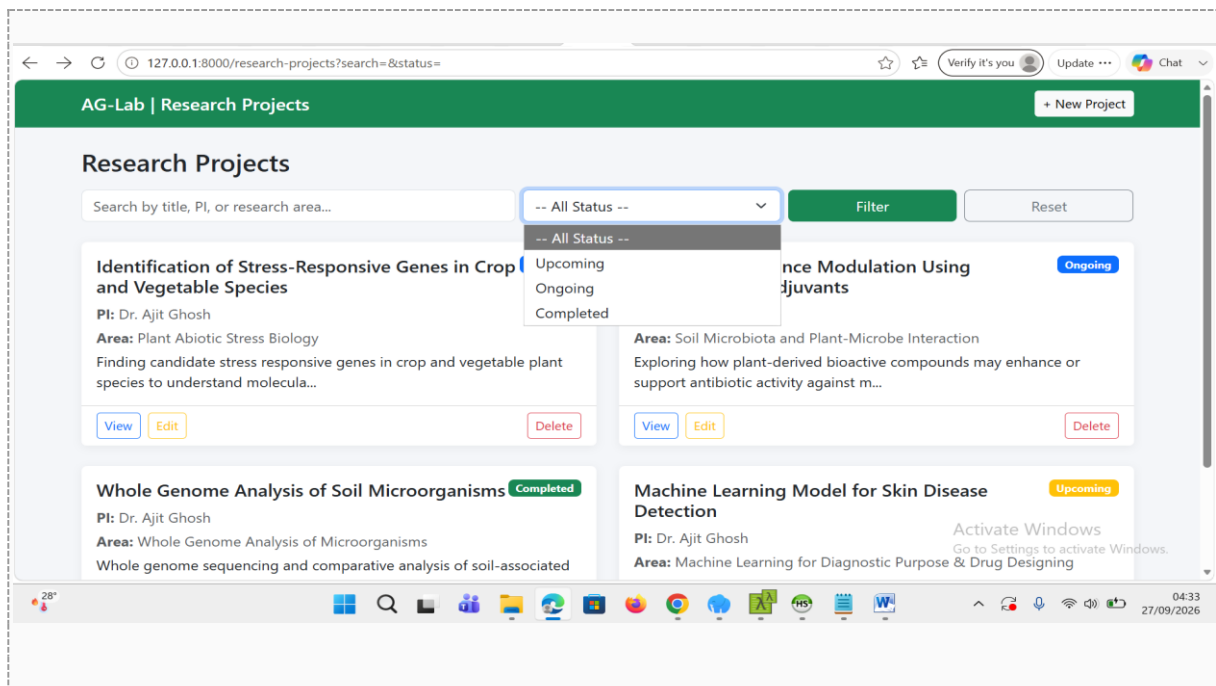


Figure 6.2(a): The Research Projects index/list page, with the search box and status filter dropdown

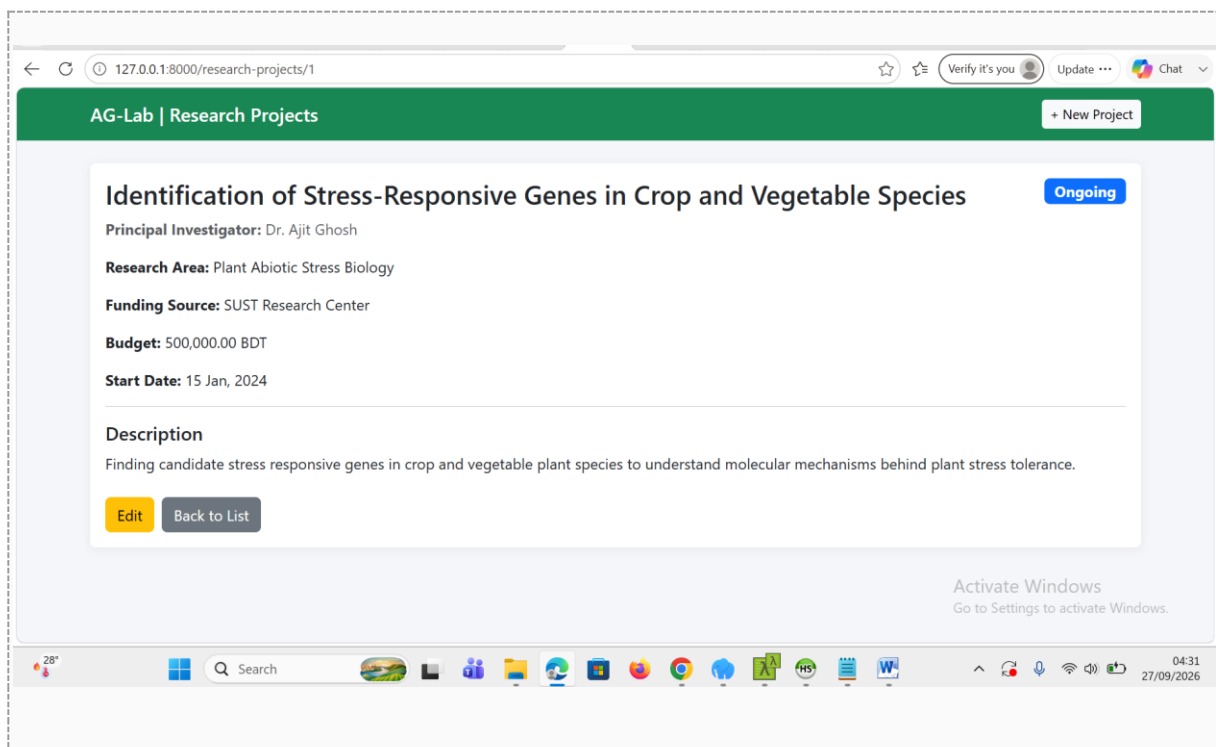


Figure 6.2(b): The details (show) page for an ongoing project

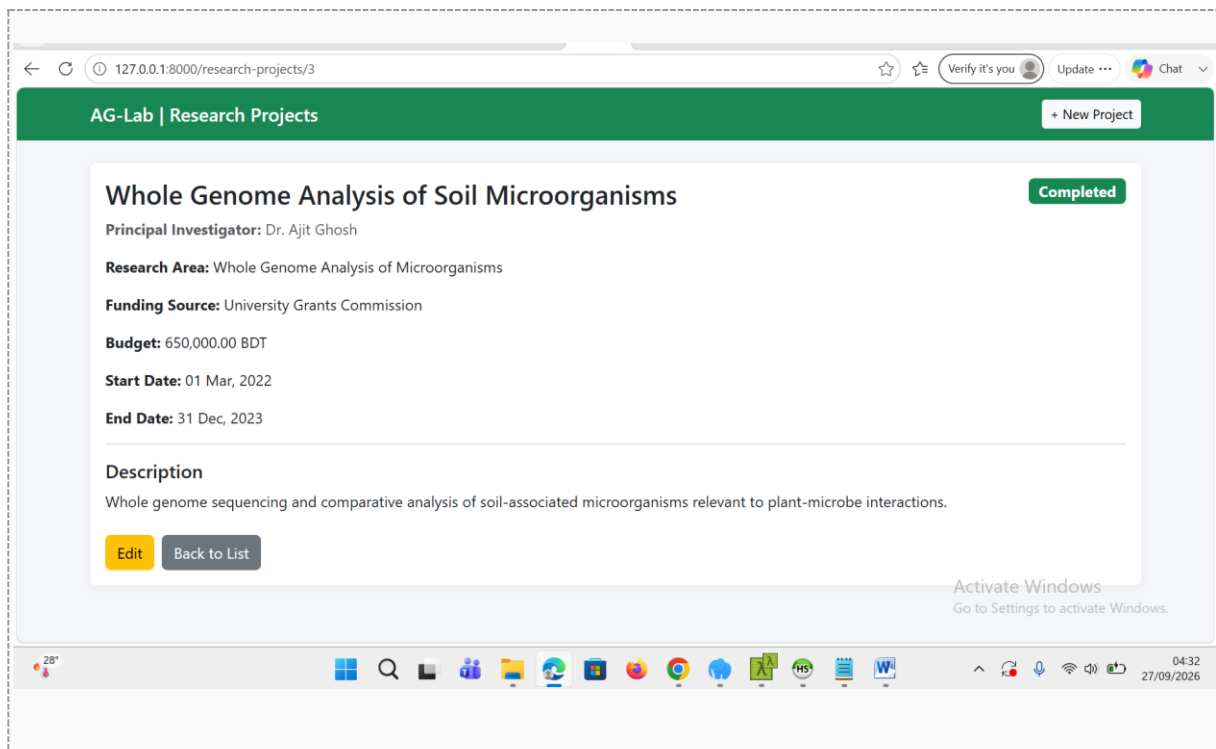


Figure 6.2(c): The details (show) page for a completed project, showing the additional End Date field

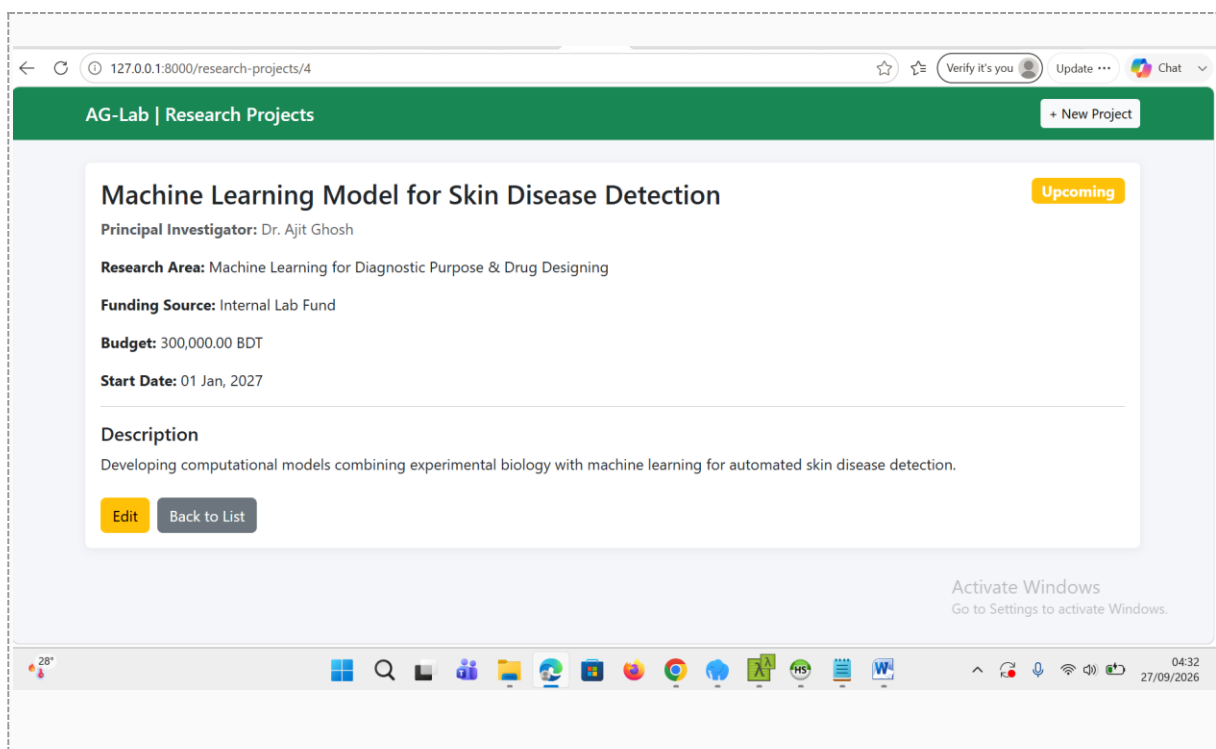


Figure 6.2(d): The details (show) page for an upcoming project, showing the yellow 'Upcoming' status badge

6.3 Update

Objective: to modify an existing project's details.

Clicking 'Edit' on any project opens the same form partial (_form.blade.php) used for creating a project, but pre-filled with that project's current values. Submitting the form runs it through the same validateData() rules and calls \$researchProject->update(\$validated), after which the user is redirected back to the list with a success message.

Figure 6.3: The 'Edit Research Project' form, pre-filled with an existing project's data

6.4 Delete

Objective: to permanently remove a project record.

Each card on the index page has a 'Delete' button inside a small form that submits a DELETE request. A JavaScript confirm() dialog ('Are you sure you want to delete this project?') is shown before the form actually submits, to guard against accidental clicks. If the project has an uploaded image, the destroy() action removes that file from storage before deleting the database row itself.

7. Additional Features

7.1 Search and Status Filter

The index() controller method builds its query conditionally: when a search keyword is present, it matches against title, principal_investigator and research_area using a LIKE query; when a status is selected, it adds a where('status', ...) clause. Both filters can be combined, and withQueryString() keeps the current search/filter values in the pagination links.

```
if ($request->filled('search')) {
    $query->where(function ($q) use ($search) {
        $q->where('title', 'like', "%{$search}%")
        ->orWhere('principal_investigator', 'like', "%{$search}%")
    })
}
```

```

        ->orWhere('research_area', 'like', "%{$search}%");
    });
}
if ($request->filled('status')) {
    $query->where('status', $request->status);
}

```

7.2 Automatic Slug Generation

Every project is given a unique, URL-friendly slug automatically derived from its title (for example, 'Whole Genome Analysis of Soil Microorganisms' becomes whole-genome-analysis-of-soil-microorganisms). If two projects share the same title, a numeric suffix (-1, -2, ...) is appended so the slug column's unique constraint is never violated.

7.3 Image Upload

The create and edit forms accept an optional cover image (jpg, jpeg, png or webp, up to 2 MB). Uploaded files are stored on Laravel's public disk under storage/app/public/research_projects and referenced from the database by their relative path. When a project's image is replaced or the project is deleted, the old file is removed from disk to avoid leaving orphaned files behind.

7.4 Pagination

The project list is paginated at eight records per page using Eloquent's paginate(8), and Laravel automatically renders Bootstrap-styled 'next / previous' page links at the bottom of the list.

7.5 Form Validation

All input is validated on the server before it reaches the database: required fields (title, description, principal investigator, start date) must be present, the status must be one of the three allowed values, the end date (if given) cannot fall before the start date, and the budget must be a non-negative number. Validation errors are automatically redirected back to the form together with the old input, so the user never has to re-type the whole form after a mistake.

8. Version Control and Deployment (Git & GitHub)

The completed project was placed under Git version control and pushed to a remote repository on GitHub, so the source code is backed up and can be shared or cloned onto another machine.

```

git init
git add .
git config --global user.email "manikdey.cs@gmail.com"
git config --global user.name "Manikdey96"
git commit -m "Add Research Projects CRUD module"
git branch -M main
git remote add origin https://github.com/Manikdey96/aglab-research-projects-.git
git push -u origin main

```

Two practical issues came up while pushing the repository for the first time, both common for a first-time Git setup on a new machine:

- Git refused to record the first commit with the error 'Author identity unknown', because no name/email had yet been configured on this machine. This was resolved by running `git config --global user.email` and `git config --global user.name` once, after which the commit succeeded.
- Because the first commit had not actually been created, the following `git push` failed with 'src refs/heads/main does not match any'. Re-running the commit before pushing again resolved this.

GitHub authentication was handled through Git Credential Manager: on the first push, a 'CredentialHelperSelector' dialog appeared, and choosing the 'manager' option opened a browser window to sign in to GitHub and authorize the push.

```

C:\laragon\www\aglab-research
> git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command: <git>
hint:
hint:   git branch -m <name>
Initialized empty Git repository in C:/laragon/www/aglab-research/.git/
C:\laragon\www\aglab-research(master)
> git add .
C:\laragon\www\aglab-research(master)
> git commit -m "Add Research Projects CRUD module"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'brdba@admin.(none)')

C:\laragon\www\aglab-research(master)
> git branch -M main
  
```

The screenshot also shows the Git Credential Manager dialog box with the following list of project files:

Section	Section	Section
0.0	0.0	0.0
0.1	0.1	0.1
0.2	0.2	0.2
0.3	0.3	0.3
0.4	0.4	0.4
0.5	0.5	0.5
0.6	0.6	0.6
0.7	0.7	0.7
0.8	0.8	0.8
0.9	0.9	0.9
1.0	1.0	1.0
1.1	1.1	1.1
1.2	1.2	1.2
1.3	1.3	1.3
1.4	1.4	1.4
1.5	1.5	1.5
1.6	1.6	1.6
1.7	1.7	1.7
1.8	1.8	1.8
1.9	1.9	1.9
2.0	2.0	2.0
2.1	2.1	2.1
2.2	2.2	2.2
2.3	2.3	2.3
2.4	2.4	2.4
2.5	2.5	2.5
2.6	2.6	2.6
2.7	2.7	2.7
2.8	2.8	2.8
2.9	2.9	2.9
3.0	3.0	3.0
3.1	3.1	3.1
3.2	3.2	3.2
3.3	3.3	3.3
3.4	3.4	3.4
3.5	3.5	3.5
3.6	3.6	3.6
3.7	3.7	3.7
3.8	3.8	3.8
3.9	3.9	3.9
4.0	4.0	4.0
4.1	4.1	4.1
4.2	4.2	4.2
4.3	4.3	4.3
4.4	4.4	4.4
4.5	4.5	4.5
4.6	4.6	4.6
4.7	4.7	4.7
4.8	4.8	4.8
4.9	4.9	4.9
5.0	5.0	5.0
5.1	5.1	5.1
5.2	5.2	5.2
5.3	5.3	5.3
5.4	5.4	5.4
5.5	5.5	5.5
5.6	5.6	5.6
5.7	5.7	5.7
5.8	5.8	5.8
5.9	5.9	5.9
6.0	6.0	6.0
6.1	6.1	6.1
6.2	6.2	6.2
6.3	6.3	6.3
6.4	6.4	6.4
6.5	6.5	6.5
6.6	6.6	6.6
6.7	6.7	6.7
6.8	6.8	6.8
6.9	6.9	6.9
7.0	7.0	7.0
7.1	7.1	7.1
7.2	7.2	7.2
7.3	7.3	7.3
7.4	7.4	7.4
7.5	7.5	7.5
7.6	7.6	7.6
7.7	7.7	7.7
7.8	7.8	7.8
7.9	7.9	7.9
8.0	8.0	8.0
8.1	8.1	8.1
8.2	8.2	8.2
8.3	8.3	8.3
8.4	8.4	8.4
8.5	8.5	8.5
8.6	8.6	8.6
8.7	8.7	8.7
8.8	8.8	8.8
8.9	8.9	8.9
9.0	9.0	9.0
9.1	9.1	9.1
9.2	9.2	9.2
9.3	9.3	9.3
9.4	9.4	9.4
9.5	9.5	9.5
9.6	9.6	9.6
9.7	9.7	9.7
9.8	9.8	9.8
9.9	9.9	9.9

Figure 8.1: The Git Credential Manager dialog shown during the first 'git push', together with the list of project files being committed.

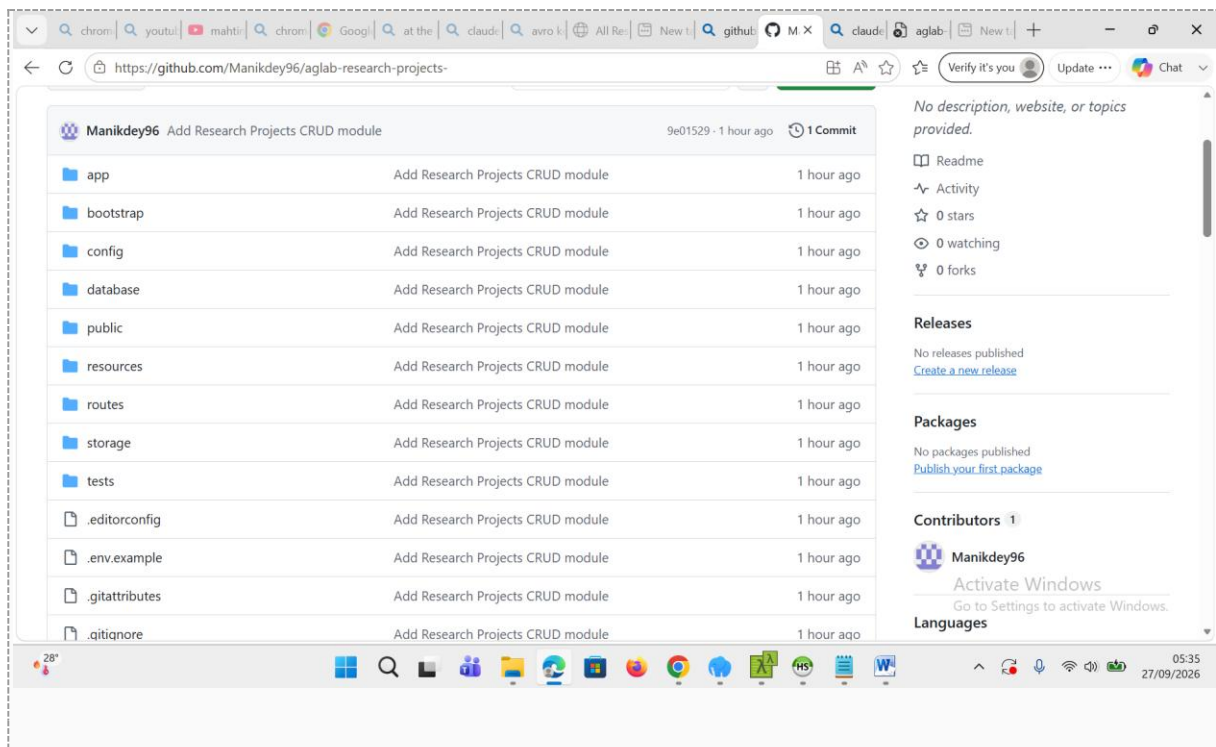


Figure 8.2: The GitHub repository page after a successful push, showing the project files.

9. Discussion

Choice of a single table. Because every attribute needed by the application belongs to exactly one project record, a single `research_projects` table was sufficient for this project, and no join or lookup tables were required. This kept the schema simple while still allowing rich filtering (by keyword and by status) at the query level.

Framework conventions versus manual code. Laravel's Eloquent ORM and its resource routing removed a large amount of boilerplate that would otherwise be needed to hand-write SQL queries and URL routing: a single `Route::resource()` line registers all seven CRUD routes, and Eloquent methods such as `create()`, `update()` and `delete()` map directly onto SQL INSERT, UPDATE and DELETE statements without the developer writing any raw SQL. This mirrors the general trade-off between low-level and high-level tools — writing raw PHP/SQL by hand offers more direct control, while a framework trades some of that control for a large reduction in development time.

Data integrity safeguards. Several small design decisions protect the integrity of the stored data: the unique constraint on slug prevents duplicate URLs, the enum on status restricts it to exactly three valid values, the `after_or_equal:start_date` validation rule prevents an end date earlier than the start date, and deleting a project's image file whenever the record is deleted or replaced prevents orphaned files from accumulating in storage.

Environment setup challenges. As with most first-time project setups, a meaningful part of the effort went into configuring the local environment and Git correctly rather than writing the CRUD logic itself. The Git identity error and the resulting failed push (Section 8) were the main obstacles encountered, and both were resolved by configuring the local Git identity once before committing again.

10. Conclusion

This project demonstrated a complete, small-scale database-backed web application: designing a normalized `research_projects` table, building a Laravel MVC application around it with the Eloquent ORM, and implementing full Create, Read, Update and Delete functionality through Blade views, together with search, status filtering, pagination, image upload and server-side validation.

The CRUD operations were verified directly against the running application, and the project was version-controlled with Git and published to a public GitHub repository, making the complete source code available for review or reuse.

As a possible extension, the single `research_projects` table could be broken into related tables (for example, a separate `investigators` table linked with a foreign key, or a `project_publications` table), so that Eloquent relationships (`hasMany` / `belongsTo`) and SQL JOIN queries could be exercised in addition to the single-table CRUD operations implemented here.